Relational Database Concepts

Tables, relationships, normalization (1NF–3NF); when to use RDBMS || Day 2

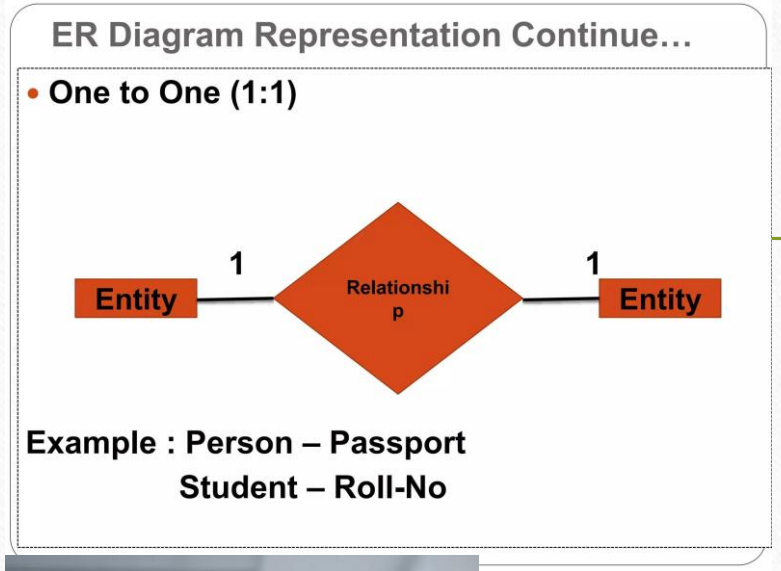
Rahul Guha || SME || Java FS Training || HCL

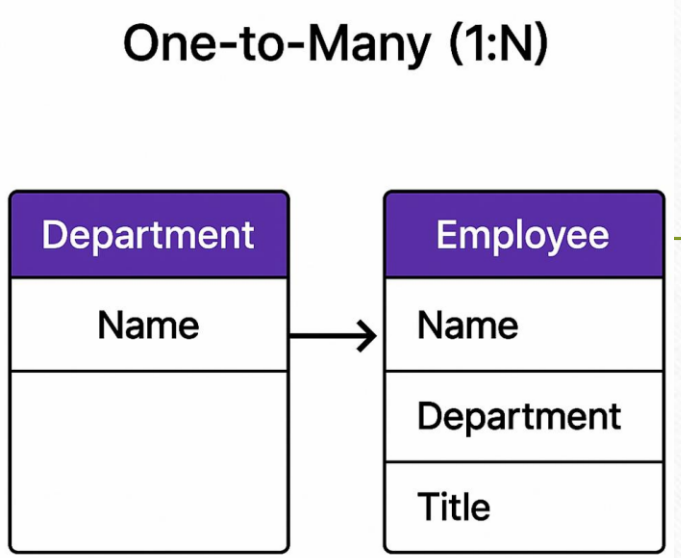
5. SQL & Database

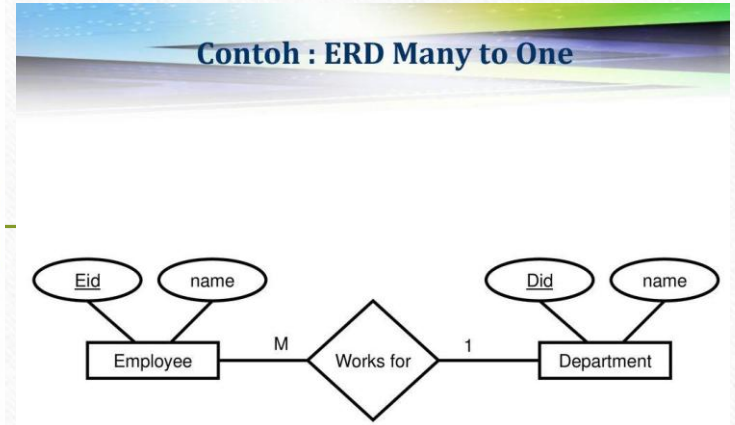
Cardinality in ER Model (DBMS)

Cardinality defines how many instances of one entity can be associated with instances of another entity through a relationship.
For example, how many employees can belong to one department?
1. Types of Cardinality
1. One-to-One (1:1)
One entity is related to at most one entity of another type, and vice versa.
Example: Person — Passport- One person has one passport.- One passport belongs to one person.
2. One-to-Many (1:N)
One entity can be related to multiple entities of another type.
Example: Department — Employee- One department has many employees.- Each employee belongs to one department.
3. Many-to-One (N:1)
Multiple entities are related to one entity.
Example: Employee — Department- Many employees can work in one department.- From the employee's perspective, the relationship is N:1.
4. Many-to-Many (M:N) Multiple entities on both sides can be associated with multiple entities on the other side.
Example: Student — Course- One student can enroll in many courses.- One course can have many students.

Cardinality in ER Model (DBMS)

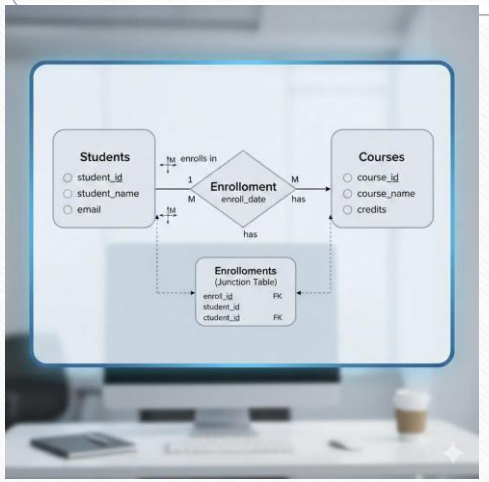






Notation guide:

- •|| — exactly one
- o{ — zero or many
- •o| — zero or one
- |{ — one or many



2. Cardinality comparison table

Cardinality	Meaning	Example
1:1	One to one	Person – Passport
1:N	One to many	Department – Employee
N:1	Many to one	Employee – Department
M:N	Many to many	Student – Course

Functional Dependency in DBMS

A Functional Dependency (FD) is a relationship between attributes (columns) in a relational database where one attribute, or a group of attributes, uniquely determines another attribute.
It is an important concept in database normalization (1NF, 2NF, 3NF, and BCNF).

1. Basic concept

Functional dependency is represented as: X -> Y

X: Determinant — the attribute or set of attributes that determines another attribute.

Y: Dependent — the attribute whose value is determined by X.

In simple words, if we know the value of X, we can determine the value of Y uniquely.

Example: Student table

Functional dependencies:
Student_ID -> Student_Name
Student_ID-> Department

Student_ID	Student_Name	Department
101	Rahul	CSE
102	Amit	IT
103	Priya	CSE
104	Neha	AI

Knowing the Student_ID determines the student's name and department.
However, Department → Student_Name is not a valid functional dependency because multiple students can belong to the same department.

Database Normalization in DBMS

Normalization is the process of organizing database tables to reduce data redundancy, prevent data anomalies, and improve data integrity.
It is based on keys and functional dependencies and is commonly applied through First Normal Form (1NF), Second Normal Form (2NF), Third Normal Form (3NF), and Boyce–Codd Normal Form (BCNF).
1. Why do we need normalization?
Consider this unnormalized student enrollment table:

Student_ID	Student_Name	Course_ID	Course_Name	Instructor
101	Rahul	C01	Java	Amit
101	Rahul	C02	DBMS	Priya
102	Neha	C01	Java	Amit

Notice that Student_Name, Course_Name, and Instructor are repeated. This creates three common problems:
Update anomaly: If the Java instructor changes, multiple rows may need updating.
Insertion anomaly: You may be unable to store a new course until a student enrolls.
Deletion anomaly: Deleting the last enrollment in a course may also remove the only record of that course.

Types of normalization

1. First Normal Form (1NF)

Each cell contains a single, atomic value. There are no repeating groups or lists inside a cell.
Before 1NF: Student 101 has Java, DBMS in one Course column.
After 1NF: Store one course per row.

2. Second Normal Form (2NF)

The table must be in 1NF, and every non-prime attribute must depend on the whole of every candidate key, not just a proper subset.
Example: Student_ID → Student_Name, while Course_ID → Course_Name. These are partial dependencies when the key is (Student_ID, Course_ID).

3. Third Normal Form (3NF)

The table must be in 2NF and have no disallowed transitive dependency of non-prime attributes on candidate keys.
Example: Student_ID → Dept_ID → Dept_Name. Store department details in a separate Department table.

4. Boyce–Codd Normal Form (BCNF)

For every non-trivial functional dependency X → Y, X must be a superkey.
BCNF is stricter than 3NF and addresses certain anomalies that can remain in 3NF.

Types of normalization

Student_ID	Student_Name	Course_ID	Course_Name	Instructor
101	Rahul	C01	Java	Amit
101	Rahul	C02	DBMS	Priya
102	Neha	C01	Java	Amit

Step 1 — Convert the table to 1NF

Assume each row represents one student's enrollment in one course.

Rule: Each column must contain atomic values, with no repeating groups or multiple values in a single cell.

Suppose the original data instead stores multiple courses in one row:
This structure violates 1NF because Courses contains multiple values.

Student_ID	Student_Name	Courses
101	Rahul	Java, DBMS
102	Neha	Java

Now each cell contains one value, and each enrollment has its own row.

Types of normalization

Student_ID	Student_Name	Course_ID	Course_Name	Instructor
101	Rahul	C01	Java	Amit
101	Rahul	C02	DBMS	Priya
102	Neha	C01	Java	Amit

Second Normal Form (2NF)

1. First Normal Form (1NF)

Each cell contains a single, atomic value. There are no repeating groups or lists inside a cell.

Before 1NF: Student 101 has Java, DBMS in one Course column.

Rule: The table must be in 1NF, and every non-key attribute must depend on the whole candidate key, not just part of a composite key.

Our composite key is:(Student_ID, Course_ID)

A composite key in a Database Management System (DBMS) is a candidate key or primary key that consists of two or more columns (attributes) that together uniquely identify a specific record in a table.

But:-

Student_ID → Student_Name

Course_ID → Course_Name, Instructor

These are partial dependencies because each set of details depends on only one part of the composite key.

Now, **Decompose the table- Code**

Types of normalization

First, separate student information:

```
CREATE TABLE Student_2NF (
    Student_ID INT PRIMARY KEY,
    Student_Name VARCHAR(50)
);

INSERT INTO Student_2NF VALUES
(101, 'Rahul'),
(102, 'Neha');
```

Then create an intermediate enrollment table that still contains course details:

```
CREATE TABLE Enrollment_2NF (
    Student_ID INT,
    Course_ID VARCHAR(10),
    Course_Name VARCHAR(50),
    Instructor VARCHAR(50),
    PRIMARY KEY (Student_ID, Course_ID),
    FOREIGN KEY (Student_ID)
        REFERENCES Student_2NF(Student_ID)
);
```

```
INSERT INTO Enrollment_2NF
VALUES(101, 'C01', 'Java', 'Amit'),(101, 'C02',
'DBMS', 'Priya'),(102, 'C01', 'Java', 'Amit');
SELECT * FROM Student_2NF;
SELECT * FROM Enrollment_2NF;
```

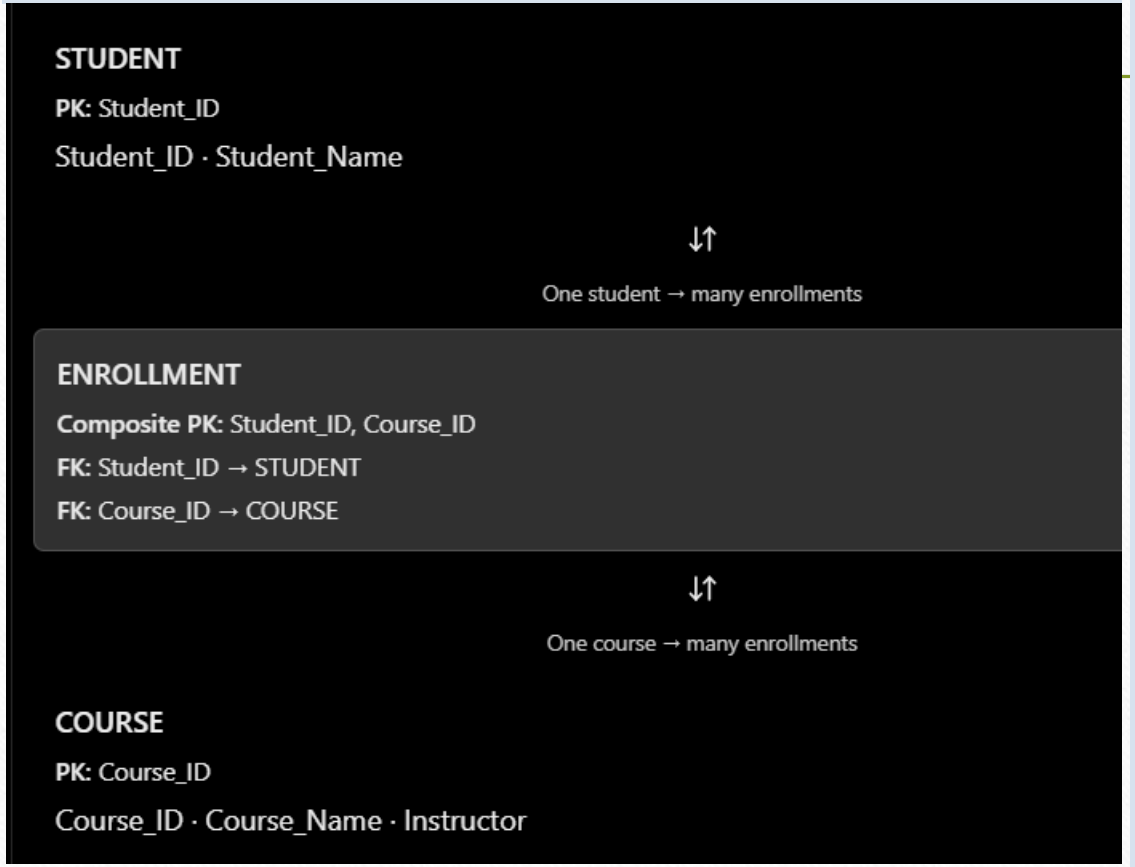
Student names are now stored separately. However, Course_Name and Instructor still depend only on Course_ID, so Enrollment_2NF is not fully in 2NF. This intermediate step helps demonstrate why we need another decomposition.

Third Normal Form (3NF)Rule:

The table must be in 2NF, and non-key attributes must not depend transitively on a candidate key through another non-key attribute.

Types of normalization

For our data, course information belongs in its own table. The final design uses three tables.



3NF SQL code

```
CREATE TABLE Student (
   Student_ID INT PRIMARY KEY,
   Student_Name VARCHAR(50) NOT NULL
);
CREATE TABLE Course (
   Course_ID VARCHAR(10) PRIMARY KEY,
   Course_Name VARCHAR(50) NOT NULL,
   Instructor VARCHAR(50) NOT NULL
);
CREATE TABLE Enrollment (
   Student_ID INT,
   Course_ID VARCHAR(10),
   PRIMARY KEY (Student_ID, Course_ID),

   FOREIGN KEY (Student_ID)
      REFERENCES Student(Student_ID),

   FOREIGN KEY (Course_ID)
      REFERENCES Course(Course_ID)
);
INSERT INTO Student VALUES
(101, 'Rahul'),
(102, 'Neha');
INSERT INTO Course VALUES
('C01', 'Java', 'Amit'),
('C02', 'DBMS', 'Priya');
INSERT INTO Enrollment VALUES
(101, 'C01'),
(101, 'C02'),
(102, 'C01');
```

Types of normalization

Retrieve the original result using JOIN

Normalization does not mean losing information. We can reconstruct the original view of the data:

```
SELECT
    s.Student_ID,
    s.Student_Name,
    c.Course_ID,
    c.Course_Name,
    c.Instructor
FROM Student s
INNER JOIN Enrollment e
   ON s.Student_ID = e.Student_ID
INNER JOIN Course c
   ON e.Course_ID = c.Course_ID
ORDER BY s.Student_ID, c.Course_ID;
```

Student_ID	Student_Name	Course_ID	Course_Name	Instructor
101	Rahul	C01	Java	Amit
101	Rahul	C02	DBMS	Priya
102	Neha	C01	Java	Amit

Boyce–Codd Normal Form (BCNF)

Rule: **For every non-trivial functional dependency X->Y,X must be a superkey.**

Under the dependencies specified for our student-course example, the three-table 3NF design also satisfies BCNF. No further decomposition is needed.

Types of normalization

To understand a case where 3NF does not guarantee BCNF, consider a different relation:Teaching(Student_ID, Course_ID, Instructor)

Suppose the business rules say:

- Each (Student_ID, Course_ID) pair identifies an instructor.

- Each instructor is assigned to exactly one course.

- Multiple students can have the same instructor.

The functional dependencies include:-

(Student_ID, Course_ID) → Instructor

Instructor → Course_ID

If Instructor is not a superkey, the second dependency violates BCNF. Decompose into:

```
CREATE TABLE Instructor_Course (
    Instructor VARCHAR(50) PRIMARY KEY,
    Course_ID VARCHAR(10) NOT NULL
);
CREATE TABLE Student_Instructor (
    Student_ID INT,
    Instructor VARCHAR(50),
    PRIMARY KEY (Student_ID, Instructor),
    FOREIGN KEY (Instructor)
        REFERENCES Instructor_Course(Instructor)
);
```

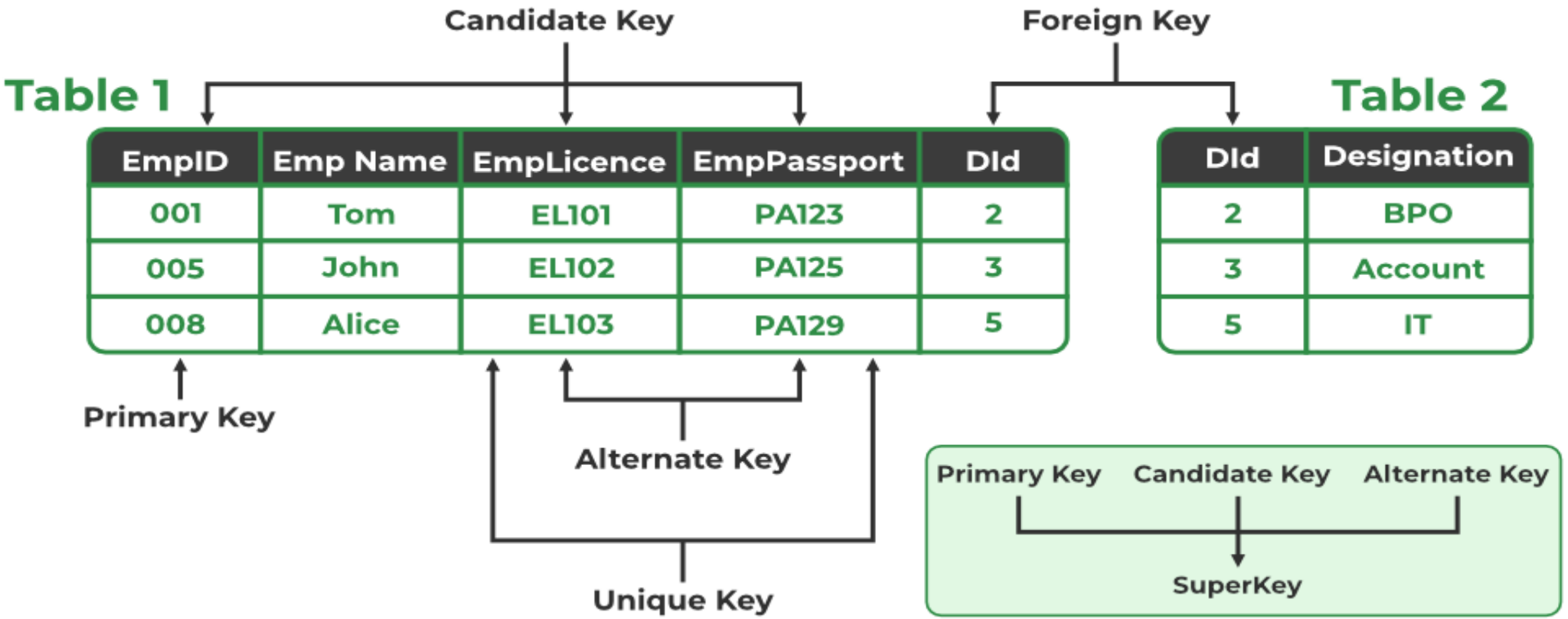
This example is only valid if the stated instructor assignment rules actually hold. Normalization must be based on real functional dependencies, not just the sample rows.

Summary of all normalization forms

Normal Form	Main requirement	In our example
1NF	Atomic values; no repeating groups	One course per row
2NF	No partial dependency on a composite key	Separate student details from enrollment
3NF	No transitive dependency of non-key attributes	Separate course details into Course
BCNF	Every determinant must be a superkey	The final three-table design already satisfies BCNF under our assumptions

INNER JOIN : Purpose: Display only students who have a matching enrollment and course.
LEFT JOIN : Purpose: Display all students, including students who have not enrolled in any course.
RIGHT JOIN : Purpose: Display all courses, including courses without enrolled students.
FULL OUTER JOIN : Purpose: Display all students and all courses, including unmatched records.
MySQL does not directly support FULL OUTER JOIN.
You can simulate it using LEFT JOIN, UNION ALL, and a second LEFT JOIN.
GROUP BY : Purpose: Group rows and calculate a result for each group.
HAVING : Purpose: Filter groups after GROUP BY has calculated the aggregate values.

Summary of all normalization forms



Database Normalization – Step by Step with Example

Example: Student Course Enrollment System

Goal: Remove redundancy and avoid anomalies by applying 1NF, 2NF, 3NF and BCNF.

1 Unnormalized Table (UNF)

- This table contains repeating groups and multiple values in a single column.
- Violates 1NF (values are not atomic).

Student_ID	Student_Name	Department	Courses (CourseID, CourseName, Instructor)
101	Rahul	CSE	C01-Java-Amit, C02-DBMS-Priya
102	Neha	IT	C01-Java-Amit, C03-Python-Rohit
103	Amit	CSE	C02-DBMS-Priya
104	Priya	AI	C01-Java-Amit, C04-AI-Sunil

Problems:

- Multiple course values in one column (repeating group)
- Not in 1NF (values are not atomic)
- Redundancy (same course and instructor repeated)
- Update, insertion and deletion anomalies

2 First Normal Form (1NF)

- Make each value atomic (single value per cell).
- Remove repeating groups.
- Still one table, but one course per row.

Student_ID	Student_Name	Department	Course_ID	Course_Name	Instructor
101	Rahul	CSE	C01	Java	Amit
101	Rahul	CSE	C02	DBMS	Priya
102	Neha	IT	C01	Java	Amit
102	Neha	IT	C03	Python	Rohit
103	Amit	CSE	C02	DBMS	Priya
104	Priya	AI	C01	Java	Amit
104	Priya	AI	C04	AI	Sunil

Now in 1NF:

- All values are atomic.
- No repeating groups.
- Primary key can be **(Student_ID, Course_ID)**

3 Second Normal Form (2NF)

- Table must be in 1NF.
- Remove partial dependency (non-key attributes should depend on the whole composite key).
- Separate data that depends only on part of the key.

Students

Student_ID (PK)	Student_Name	Department
101	Rahul	CSE
102	Neha	IT
103	Amit	CSE
104	Priya	AI

Courses

Course_ID (PK)	Course_Name	Instructor
C01	Java	Amit
C02	DBMS	Priya
C03	Python	Rohit
C04	AI	Sunil

Enrollments

Student_ID (PK, FK)	Course_ID (PK, FK)
101	C01
101	C02
102	C01
102	C03
103	C02
104	C01
104	C04

4 Third Normal Form (3NF)

- Table must be in 2NF.
- Remove transitive dependency (non-key attribute should not depend on another non-key attribute).
- Separate department information.

Students

Student_ID (PK)	Student_Name	Dept_ID (FK)
101	Rahul	D01
102	Neha	D02
103	Amit	D01
104	Priya	D03

Departments

Dept_ID (PK)	Dept_Name
D01	Computer Science
D02	Information Technology
D03	Artificial Intelligence

Courses

Course_ID (PK)	Course_Name	Instructor
C01	Java	Amit
C02	DBMS	Priya
C03	Python	Rohit
C04	AI	Sunil

Enrollments

Student_ID (PK, FK)	Course_ID (PK, FK)
101	C01
101	C02
102	C01
102	C03
103	C02
104	C01
104	C04

5 Boyce–Codd Normal Form (BCNF)

- Table must be in 3NF.
- For every non-trivial functional dependency X → Y, X must be a superkey.
- Remove any remaining anomalies due to overlapping functional dependencies.

Example Situation Requiring BCNF:

- Suppose each instructor teaches only one course.
- Then we have the functional dependency: **Instructor → Course_ID**
- Instructor is not a superkey, so this violates BCNF.
- Split the Courses table.

Instructors

Instructor (PK)	Course_ID (FK)
Amit	C01
Priya	C02
Rohit	C03
Sunil	C04

Courses

Course_ID (PK)	Course_Name
C01	Java
C02	DBMS
C03	Python
C04	AI

Views, Indexing & Constraint

Constraints are rules applied to columns or tables to maintain data integrity.

Constraint	Purpose
PRIMARY KEY	Uniquely identifies each record; cannot be NULL
FOREIGN KEY	Maintains relationships between tables
UNIQUE	Prevents duplicate values
NOT NULL	Prevents missing values
CHECK	Ensures values satisfy a condition
DEFAULT	Supplies a value when none is specified